

Big Data Analysis Mini-Project

Implementation Guide: Hospital Patient Vital Signs Monitoring

A practical step-by-step guide for architecture, implementation, observability, report writing, deliverables, testing, and demonstration.

Selected use case: Continuous bedside vital-sign monitoring correlated with daily pathology-lab results.

Recommended project direction: Use a Lambda-style architecture for this assignment, with a real-time streaming path for immediate monitoring and a scheduled batch/historical path for daily consolidation. The report should still compare Lambda and Kappa and explicitly discuss why Kappa was rejected.

Important: This is an academic simulation. Risk thresholds and scores should be clearly described as project-defined monitoring rules, not clinical diagnosis or medical advice.

1. What You Are Actually Building

The assignment asks for a small but complete data platform. Your system must continuously receive simulated patient vital signs, receive a simulated daily laboratory file, process the information, combine the two sources, store useful results, expose real-time information, produce a daily consolidated report, and make the pipeline observable.

The simplest mental model is:

```
Python vital simulator → Kafka → Spark Structured Streaming → real-time risk/alerts/API
Python daily lab simulator → daily file → batch/historical processing → consolidated report
Both paths → storage → serving/reporting
Airflow → orchestrates the scheduled batch/reporting workflow
Logging + metrics + health checks → across the pipeline
```

The business question is: Which patients show concerning vital-sign trends right now, and how do yesterday's lab results change the risk picture for those patients going forward?

2. Requirements You Must Satisfy

Requirement	What your project should demonstrate
Streaming source	Python script emits vital readings every few seconds; include occasional simulated abnormal spikes.
Daily batch source	Python script creates one lab-result file per simulated day.
Kafka	Producer(s), topic(s), keys, partitions, consumer/streaming ingestion.
Spark	Structured Streaming for real-time processing; meaningful transformations such as cleaning, windowing, trend calculation.
Daily/historical processing	Use the daily lab data and historical vital data to produce a consolidated report.
Storage	Queryable store; PostgreSQL is a practical choice for this project.
Serving	Real-time API endpoint and/or dashboard showing current monitoring information.
Alerts	Threshold/rule-based patient alerts and at least one pipeline-health alert.
Airflow	Schedule/orchestrate daily batch, consolidation and reporting workflow.
Observability	Structured logs, metrics and health/alert rules.
Reproducibility	README, configuration, tests where appropriate, and preferably Docker Compose.
Report + demo	8-15 page report, PDF, screenshots/results, and 5-10 minute end-to-end demo.

3. Recommended Architecture Decision: Lambda

For this assignment, a Lambda-style architecture is a strong choice because the use case has two clearly different processing needs: immediate monitoring of streaming vitals and scheduled historical/daily consolidation involving laboratory data.

Recommended conceptual architecture:

```
BEDside simulator
↓
Kafka: patient-vitals topic
↓
Spark Structured Streaming
↓
Current trend / threshold evaluation / patient alert state
↓
PostgreSQL + Real-time API

Daily lab simulator
↓
CSV/JSON daily file
```

↓
 Airflow scheduled workflow
 ↓
 Spark batch/historical consolidation
 ↓
 Join latest lab results + recent/historical vital summaries
 ↓
 PostgreSQL daily risk report / report file

Why Lambda fits: the streaming path must react within seconds, while the daily lab feed naturally belongs to scheduled batch processing. Keeping these paths explicit makes the architecture easy to demonstrate and maps directly to the assignment's two-source requirement.

Important trade-off: Lambda means two processing paths, which increases code, infrastructure and the risk that streaming and batch logic calculate different results. Your report should acknowledge this instead of pretending Lambda is free of drawbacks.

4. Lambda vs Kappa — What to Write in the Report

Criterion	Lambda	Kappa
Real-time latency	Excellent: dedicated streaming path.	Excellent: streaming-first design.
Daily lab source	Natural fit for scheduled batch processing.	Can be converted/ingested into the streaming/event model, but requires additional infrastructure.
Replay	Possible through retained Kafka data and historical storage batch path concept because of historical streaming processing model can replay.	Storage batch path concept because of historical streaming processing model can replay.
Consistency	Two paths may duplicate business logic and need reconciliation.	One processing path can reduce duplicated logic.
Complexity	Higher.	Lower in a pure streaming system.
This assignment	Strong fit because both immediate and scheduled/historical processing are required.	Possible, but requires more explanation how daily files become events and how historical data is processed.

Your report should explicitly say that Kappa was considered and rejected because the assignment gives you a natural daily batch source and asks for historical/reporting orchestration. Do not say Kappa is 'bad'; explain that it would require reshaping the batch feed into the event stream and designing replay/reprocessing carefully. Also acknowledge that Kappa could reduce duplicated processing logic and might be preferable at a different production scale or with an event-native lab feed.

5. Recommended Technology Stack

Layer	Recommended tool	Reason tied to this use case
Simulation	Python	Simple, fast way to generate continuous vitals and daily lab files; easy to control simulated time.
Ingestion	Apache Kafka	Durable, partitioned event stream suitable for continuous vital-sign events and decoupling production from processing.
Streaming processing	Apache Spark Structured Streaming	Supports windowed aggregations, filtering, enrichment and scalable streaming computations.
Batch/historical	Apache Spark batch	Reuse Spark ecosystem for daily/historical consolidation and report preparation.
Orchestration	Apache Airflow	Schedules the daily lab/consolidation/reporting workflow and exposes task status/retries/logs.
Storage	PostgreSQL	Easy to query, inspect and serve structured patient monitoring summaries in a university project.
Serving	FastAPI (or similar Python API)	Simple REST endpoint for real-time ward figures and alert state.
Observability	Structured JSON logs + Prometheus	Makes pipeline behaviour measurable and demonstrable; supports health rules.
Packaging	Docker Compose	Reproducible local environment for Kafka, Spark, PostgreSQL, Airflow and supporting services.

You do not need to use every possible technology. The goal is a coherent stack where every component has a clear responsibility.

6. Design the Data Sources

6.1 Streaming vital-sign schema

```
patient_id, heart_rate, spo2, systolic_bp, diastolic_bp, temperature, timestamp
```

Generate readings for a fixed set of simulated patients. For example, 10–20 patients is enough for a demo. Generate one reading every 2–5 seconds. Occasionally inject controlled abnormal values so the alert path can be demonstrated.

6.2 Daily lab schema

```
patient_id, test_type, result_value, reference_range, collected_at
```

Use a small but meaningful set of lab tests. Keep the reference range in the data so your processing can determine whether a result is within the simulated reference range.

6.3 Simulated clock

Choose and document a compressed clock. A practical choice is 1 simulated day = 5 real minutes. The lab simulator then creates a new daily file every five real minutes. State this clearly in the README and report.

7. Kafka Design

A simple starting design is one topic for vital events:

```
Topic: patient-vitals  
Key: patient_id  
Value: JSON vital reading  
Partitions: 3 (example; justify the number in your report)
```

Using `patient_id` as the Kafka key keeps events for the same patient ordered within the same partition. This is useful when calculating patient-level trends.

Do not claim that 'more partitions are always better'. Explain that partitions provide parallelism and scalability, while too many partitions add operational overhead. For a small academic demo, 3 partitions is enough to visibly demonstrate partitioning without unnecessary complexity.

Test and demonstrate: producer sends messages, Kafka topic receives them, partitions contain data, and Spark consumes the stream.

8. Streaming Processing Design

The streaming pipeline should do more than copy records. A good processing sequence is:

```
Kafka → parse JSON → validate/clean → watermark/timestamp handling → 5-minute patient window → calculate trend indicators  
→ threshold checks → risk state → write results to PostgreSQL / expose to API
```

Possible derived fields:

- average heart rate over the recent window
- minimum SpO2 over the recent window
- maximum temperature over the recent window
- number of abnormal readings in the window
- direction of a trend (stable / increasing / decreasing) using a simple documented rule
- vital risk points based on project-defined thresholds
- current risk category such as NORMAL / WATCH / CONCERNING

Keep the rules simple and explainable. This project is assessing data engineering, not clinical diagnosis. Clearly label thresholds as simulated academic rules.

9. Daily Lab and Consolidation Processing

The daily workflow should take the lab file for the completed simulated day and combine it with stored vital summaries/history.

Airflow DAG

1. Check expected lab file exists
2. Validate file schema and row count
3. Load lab data
4. Run Spark batch/historical consolidation
5. Join lab results to patient vital summaries
6. Calculate final daily risk indicators
7. Write consolidated results to PostgreSQL
8. Generate CSV/HTML/PDF-style report artifact if desired
9. Record success/failure metrics

The key join is patient_id. The report should make the relationship visible: recent vital trend + latest lab findings → consolidated risk indicator.

10. Airflow Design

Airflow is the workflow manager, not the second-by-second monitoring engine. Use it for the daily path.

Example DAG tasks:

Task	Purpose
wait_for_lab_file	Check that the simulated day's lab file has arrived.
validate_lab_file	Check required columns, file readability and basic data quality.
run_batch_processing	Run Spark batch/historical consolidation.
load_report_results	Ensure the consolidated results are in PostgreSQL.
generate_daily_report	Create a human-readable daily report.
health_check	Record that the daily workflow completed successfully.

Use retries for temporary failures. Make the DAG idempotent where practical: rerunning a day should not blindly duplicate the same report rows.

11. PostgreSQL Storage Design

A practical schema can contain separate raw/processed/serving tables. Keep the design understandable rather than creating dozens of tables.

Table	Purpose	Example fields
vital_readings	Optional recent/raw vital records or curated stream output.	patient_id, timestamp, HR, SpO2, BP, temperature
patient_current_status	Latest serving state for the API.	patient_id, latest_timestamp, risk_status, risk_score, trend fields
lab_results	Daily laboratory records.	patient_id, test_type, result_value, reference_range, collected_at
daily_patient_risk	Consolidated report output.	simulated_day, patient_id, vital_summary, lab_summary, risk_status
pipeline_metrics	Optional application-level metrics snapshot.	timestamp, component, metric, value

The real-time API should read from patient_current_status or an equivalent serving table rather than directly querying the raw Kafka stream on every request.

12. Serving/API

A minimal API is enough. Example endpoints:

```
GET /health
GET /api/patients
GET /api/patients/{patient_id}
GET /api/alerts
GET /api/metrics
```

The dashboard can be simple. If time is limited, a clean API plus a small HTML/React dashboard is sufficient. The important thing is that the output answers the business question.

13. Define a Simple, Explainable Risk Rule

Do not make the risk model unnecessarily complicated. For example, define project rules such as:

Indicator	Example simulated rule	Example points
SpO2	Below the project's defined threshold	+2
Heart rate	Outside the project's defined range	+1
Temperature	Above the project's defined range	+1
Blood pressure	Outside the project's defined range	+1
Abnormal lab	Result outside supplied reference range	+1 or +2
Deteriorating trend	Repeated worsening readings in the recent window	+1

Then define categories such as NORMAL, WATCH and CONCERNING. These numbers are examples only; choose and justify your own simulated rules. Avoid presenting them as clinical guidelines.

14. Observability — A Full 10-Mark Strategy

Observability should be designed from the beginning, not added on the final day.

Area	Measure/log	Why it matters
Vital producer	events generated, send errors, last event timestamp	Detect source failure.
Kafka	records, partition, consumer lag	Detect ingestion/consumption problems.
Spark	records processed, batch duration, failed records, stream state	Detect processing failures and slowdowns.
Lab source	last file timestamp, rows, schema errors	Detect missing/invalid daily feed.
PostgreSQL	insert/update errors, write latency	Detect storage failure.
API	request count, response latency, 5xx count	Detect serving problems.
Business metrics	patients monitored, alerts, concerning patients	Show whether the system is producing useful output.

Use structured JSON logs. Every log should contain fields such as timestamp, component, event, severity and relevant identifiers. A simple health rule can be: alert if no vital event is received for 30 seconds. Another useful rule is: alert if the expected daily lab file has not arrived by the simulated-day deadline.

15. Recommended Implementation Order

Do not build the entire project at once. Build one working slice at a time.

Phase	Build	Definition of done
1	Repository + Docker skeleton	Services start and README explains setup.

Phase	Build	Definition of done
2	Vital simulator	Python continuously emits valid JSON readings and abnormal spikes.
3	Kafka	Topic, key, partitions and producer work; messages are visible.
4	Spark streaming	Spark consumes Kafka and produces calculated patient/window metrics.
5	PostgreSQL	Processed current patient status is stored and queryable.
6	API	Current patient status and alerts can be queried.
7	Daily lab simulator	A new lab file is created once per simulated day.
8	Airflow	DAG detects file, runs processing and generates daily result.
9	Consolidation	Vital summaries + latest labs produce daily risk report.
10	Observability	Structured logs, metrics and at least one alert work.
11	Tests + failure demos	Invalid data, missing feed and restart cases are handled/documented.
12	Documentation + report + demo	README, PDF report, screenshots and demo sequence complete.

16. Suggested Repository Structure

```
hospital-big-data-pipeline/ ├── README.md ├── docker-compose.yml ├── .env.example ├── config/ | ├── app.yaml | └──
thresholds.yaml ├── simulators/ | ├── vital_producer/ | └── lab_batch/ ├── kafka/ | └── init/ ├── spark/ | ├── streaming/ | ├──
batch/ | └── common/ ├── airflow/ | └── dags/ ├── api/ ├── database/ | └── init/ ├── observability/ | ├── metrics/ | └── alerts/
└── tests/ ├── reports/ | └── sample/ └── docs/ └── architecture/
```

Keep configuration such as Kafka bootstrap address, database URL, simulated-day duration and thresholds outside the source code where practical.

17. Testing Checklist

Test	Expected result
Valid vital event	Accepted and processed.
Missing required field	Rejected/logged as invalid; pipeline remains healthy.
Invalid numeric value	Handled according to documented validation rule.
Abnormal spike	Produces the expected simulated alert/risk indicator.
Multiple readings for same patient	Correct window/trend aggregation.
Missing patient lab	Patient remains in report with a documented 'lab unavailable' state.
Out-of-range lab	Lab abnormal indicator appears in consolidated output.
Missing daily lab file	Airflow task retries/fails clearly and health alert is raised.
Stop vital producer	No-data health rule eventually alerts.
Restart producer	Pipeline recovers and events resume.
API unavailable/DB unavailable	Failure is logged and visible in health/metrics.

18. How to Write the 8-15 Page Technical Report

Section	What to include
1. Introduction	Problem, motivation and scope.
2. Business requirements	Streaming monitoring, daily labs, consolidation, alerts and reporting.
3. Architecture decision	Lambda vs Kappa, latency, replay, cost, consistency, trade-offs, rejected alternative.

Section	What to include
4. Architecture	Complete diagram and explanation of each data flow.
5. Technology stack	Kafka, Spark, Airflow, PostgreSQL, API, Docker and observability; justify each against the use case.
6. Data design	Schemas, Kafka topic/key/partitions, simulated clock and sample records.
7. Processing	Cleaning, windowing, trend calculations, lab join and risk rules.
8. Storage/serving	Database schema, API endpoints and report/dashboard.
9. Observability	Logs, metrics, health checks and alert behaviour.
10. Results	Screenshots, sample report, API output, Kafka/Spark/Airflow evidence.
11. Limitations/trade-offs	Simulation limits, thresholds, scale, failure modes and production improvements.
12. Conclusion	What was implemented and whether the business question is answered.

A strong report does not just describe what you built. It explains why each design decision was made and what trade-off it creates.

19. Direct Strategy for the 100-Mark Rubric

Rubric	Marks	How to secure the marks
Architecture decision	20	Give a real Lambda/Kappa comparison; discuss latency, replay, cost, consistency; justify the chosen architecture.
Technology stack	10	Tie every major technology to a concrete use-case requirement.
Data ingestion	15	Reliable Python simulators, Kafka producer, topic, keys, partitions, validation and failure handling.
Processing	15	Real Spark transformations: windows, trends, cleaning, lab join, aggregation and risk indicators.
Storage/serving	10	Correct PostgreSQL schema, current-status serving, API and consolidated daily report.
Observability	10	Structured logs + metrics + health alerts + enough evidence to diagnose failures.
Report	15	Clear diagrams, decisions, screenshots, results, limitations and honest trade-offs.
Code quality	5	Modular code, README, config, comments/tests and reproducible Docker setup.

Highest priority: architecture + ingestion + processing + report = 65 marks. Do not spend most of your time making the dashboard visually fancy while leaving the pipeline or architecture explanation weak.

20. 5-10 Minute Demo Plan

Prepare a deterministic demo sequence so you do not depend on random behaviour.

Time	Show
0:00–1:00	Use case, business question and architecture diagram.
1:00–2:00	Docker Compose/services and Kafka topic/partitions.
2:00–3:30	Start vital simulator; show Kafka messages and Spark processing.
3:30–4:30	Show real-time API/dashboard and trigger an abnormal patient event.
4:30–5:30	Show structured logs and metrics; demonstrate no-data alert if feasible.
5:30–7:00	Advance simulated day; show daily lab file and Airflow DAG execution.
7:00–8:30	Show consolidated patient risk report joining vitals and labs.
8:30–10:00	Explain architecture trade-offs, limitations and production improvements.

Take screenshots while everything is working. Your report needs evidence of Kafka, Spark, Airflow, API/dashboard, database/report output and observability.

21. README Checklist

Your README should allow another person to reproduce the project without asking you questions.

- Project overview and business question
- Architecture diagram
- Lambda vs Kappa decision summary
- Technology stack
- Prerequisites
- Docker Compose startup command
- Environment/configuration instructions
- How to create Kafka topic(s) and partitions
- How to start streaming simulator
- How to trigger/advance a simulated day
- How to access API/dashboard
- How to inspect PostgreSQL results
- How to access Airflow
- How to view logs and metrics
- How to trigger and observe a health alert
- How to run tests
- Expected sample output
- Known limitations

22. Final Submission Checklist

Item	Ready?
Git repository or ZIP contains complete codebase	☐
Vital-sign Python simulator works continuously	☐
Daily lab Python simulator creates one file per simulated day	☐
Kafka topic + partitions demonstrated	☐
Spark Structured Streaming works	☐
Meaningful cleaning/windowing/trend/risk processing implemented	☐
Daily/historical consolidation implemented	☐
Patient vitals + latest labs joined	☐
PostgreSQL or selected store is queryable	☐
Real-time API/dashboard works	☐
Patient alert rules work	☐
Airflow DAG works and logs task status	☐

Item	Ready?
Structured logging implemented	☐
Metrics implemented	☐
At least one health/alert rule demonstrated	☐
README is reproducible	☐
Automated tests included where appropriate	☐
Architecture diagram included	☐
Report is 8–15 pages and includes screenshots	☐
Limitations and trade-offs discussed	☐
5–10 minute demo is rehearsed	☐

23. Common Mistakes to Avoid

- Do not make Kafka a simple one-message demo. Show continuous events, a meaningful topic/key/partition design and observable consumption.
- Do not use Spark only to copy Kafka records into PostgreSQL. Include meaningful transformation, windowing, aggregation and trend logic.
- Do not make the daily source a manual file that you create yourself during the demo. Automate its generation with the required Python script.
- Do not use Airflow as the real-time streaming engine. Airflow should orchestrate scheduled workflows.
- Do not hide the Lambda/Kappa trade-offs. The rubric specifically asks for latency, replay, cost and consistency.
- Do not claim the risk rules are medical diagnosis. State that thresholds are simulated academic rules.
- Do not leave observability until the last day. Logs and metrics are part of the assessed system.
- Do not rely on a fragile demo that only works on your laptop after manual commands. Docker Compose and a clear README reduce this risk.
- Do not spend excessive time on UI polish. The architecture, ingestion, processing and report carry most of the marks.

24. Production-Scale Improvements to Discuss

Your report should be honest about what this student project does not solve. Useful production-scale discussion points include:

- High availability for Kafka, Spark, PostgreSQL and Airflow
- Kafka replication and stronger delivery/processing guarantees
- Schema Registry and schema evolution
- Dead-letter handling for invalid events
- Data retention and replay policies
- Patient-data security, access control and encryption
- Audit logging and privacy/compliance requirements
- More robust stream/batch consistency strategy
- External monitoring/alerting infrastructure

- Horizontal scaling and partition strategy
- More rigorous clinical validation of any real hospital risk rules
- Replacing simulated sources with secure hospital sensor and laboratory integrations

25. Practical Order for Your Work From Today

If you want to finish this project reliably, follow this order:

- Step 1: Freeze the requirements and architecture decision.
- Step 2: Draw the final architecture before coding.
- Step 3: Define schemas and simulated-time rules.
- Step 4: Build the vital producer and Kafka topic/partitions.
- Step 5: Build Spark streaming and prove a patient window calculation works.
- Step 6: Add PostgreSQL and current patient status.
- Step 7: Add the API and real-time alert output.
- Step 8: Build the daily lab producer and automated daily file creation.
- Step 9: Build the Airflow DAG and batch consolidation.
- Step 10: Join lab + vital summaries and generate the daily report.
- Step 11: Add structured logging, metrics and health alerts throughout.
- Step 12: Dockerize the environment and make the README reproducible.
- Step 13: Add tests and failure scenarios.
- Step 14: Capture screenshots and sample results.
- Step 15: Write the report around your actual implementation and trade-offs.
- Step 16: Rehearse the 5-10 minute demo from a clean environment.

The most important rule is: make the pipeline work end-to-end before adding extra features. A complete simple pipeline earns more value than a partially implemented complex one.

Final Target Architecture — One-Page Summary

HOSPITAL WARD | Vital simulator Lab simulator Python Python every few sec once/day |
▼▼ KAFKA Daily CSV/JSON | ▼▼ Spark Structured Airflow DAG Streaming | ▼ Spark Batch |
▼ PostgreSQL ▼▼ Current Status Daily Risk Report | ▼▼ REST API
Dashboard/File Observability spans the entire system: logs + metrics + health checks + alerts

Success criterion: at the end of the demo, you should be able to show a vital event entering Kafka, Spark turning it into a meaningful patient-level result, the API exposing current status, the daily lab file arriving, Airflow running the consolidation, the database/report containing the joined result, and the observability layer showing that the pipeline is healthy.

This guide is based directly on the assignment requirements and rubric you provided. Use it as the implementation checklist, then tailor the exact thresholds, simulated data volume, dashboard design and architecture details to what your team can reliably demonstrate.